

## Segment-4

Transaction: 81 - consistent

A set of programmes which execute normalization for a given task.

Basic ap:

Transaction must be consistent.

Properties:

ACID

Atomicity



Either all operations of the transaction are reflected properly in the database, or none are.

Consistency



Execution of a transaction in isolation preserves the consistency of the database.

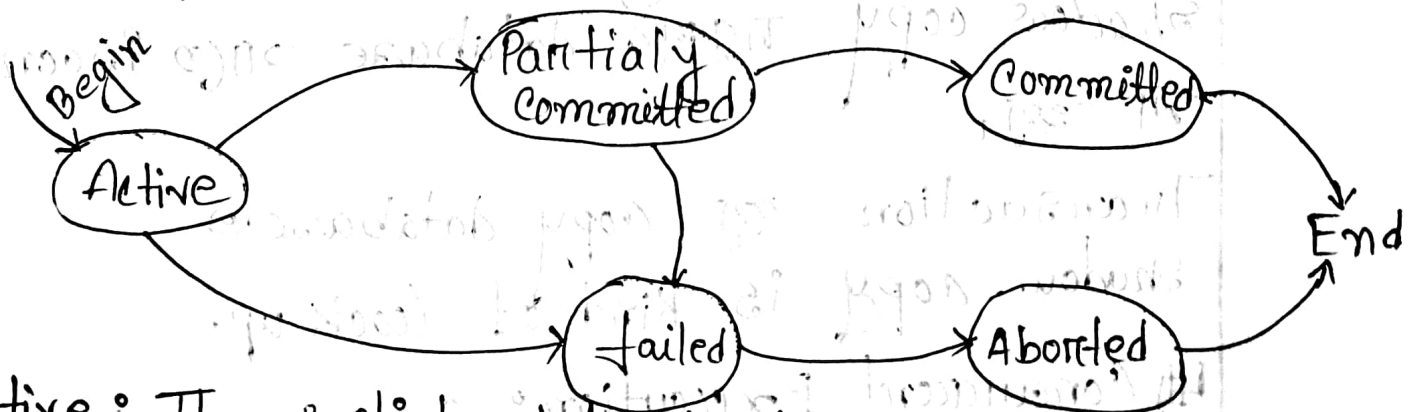
Transaction half কাজ -  
করতে পারবে, কিন্তু  
Half হতে পারবে না  
must be completed

**I = Isolation** → Even though multiple transactions may execute concurrently, the system guarantees that for every pair of transactions  $T_i$  and  $T_j$ , it appears to  $T_i$  that either  $T_j$  finished execution before  $T_i$  started, or  $T_j$  started execution after  $T_i$  finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.

(एकजन wait जानना अन्य wait को search कर रहा)

**Durability - D** → After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures. (Data Backup form द्वारा)

**State of transaction:**



**Active:** The initial state, the transaction stays in this state while it is executing.

**Partially committed:** After the final statement has been executed.

**Failed:** After the discovery that normal execution can no longer proceed.

**Aborted:** After the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.

**Committed:** After successful completion.

**Shadow Copy:** In shadow copy schema a transaction that wants to update the database first creates a complete copy of the database. All updates are done on the database leaving the original copy untouched. The shadow copy ensures consistency, atomicity, & durability.

Shadow copy द्वारा database-चा inconsistency ना-है।

Transaction-चा copy database में  
Shadow copy is kind of back up.

**Concurrent Execution:** In a multi user system multiple users can access and use the same database at one time, which is known as the concurrent execution of the database.

## Benefits:

- Increase output
- reduces the unpredictable delays.
- reduces the average response time.
- reduces costs.

### Concurrent execution

T<sub>1</sub>

read(A)

A := A - 100

write(A)

read(B)

B := B + 100

write(B)

T<sub>2</sub>

read(A)

A := A - 100

write(A)

read(B)

B := B + 100

write(B)

### Serial execution

T<sub>1</sub>

read(A)

A := A - 100

write(A)

read(B)

B := B + 100

write(B)

T<sub>2</sub>

read(A)

A := A - 100

write(A)

read(B)

B := B + 100

compatibility matrix :

|       | read | write |
|-------|------|-------|
| read  | T    | F     |
| write | F    | F     |

Problems of concurrent executions:

Dirty read:

When you see uncommitted rows in another transaction. There is no guarantee the other transaction will commit but you could return the data that was never saved to the database.

II) Fuzzy reads/non-repeated read:

A non-repeated read is when selecting the same row twice returns different result. This happens when someone update the rows in between.

III) Phantom read:

When another insert or delete rows that clause of your query. Repeated queries can return different rows.

Problem: solve क्या है एक ही way serializability

| T <sub>1</sub>                        | T <sub>2</sub>                                   |
|---------------------------------------|--------------------------------------------------|
| read (A)<br>A := A - 100<br>write (A) | serial                                           |
| read (B)<br>B := B + 100<br>write (B) | read (A)<br>A := A + 100<br>write (A) Concurrent |
|                                       | read (B)<br>B := B + 100<br>write (B) serial     |

serializability: A transaction schedule is serializable if its outcome is equal to the outcome of its transaction executed serially.

☐ Different type of serializability:

① Conflict serializability: A schedule is called conflict serializable if we can convert it into a serial schedule after swapping its non-conflicting operations.

Conflicting operations:

Two operations are said to be in conflict, if they satisfy all the following three conditions:

1. Both the operations should belong to different transactions.
2. Both the operations are working on same data item.
3. At least one of the operation is a write operation.

Ex:1 Operation  $w(x)$  of transaction  $T_1$  and operation  $R(x)$  of transaction  $T_2$  are conflicting operations because they satisfy all the three conditions mentioned above. They belong to different transactions, they are working on same data item  $x$ , one of the operation is write operation.

☐ Conflict Equivalent Schedules:

Two schedules are said to be conflict equivalent if one schedule can be converted into other schedule after swapping non-conflicting operations.

☐ Conflict Serializable check:

Let's check whether a schedule is conflict Equivalent if one schedule can be converted into other schedule serializable or not. If a schedule is conflict Equivalent to its serial schedule then it is called conflict serializable schedule.

Example of conflict serializability:

Lets consider this schedule:

| <u>T<sub>1</sub></u> | <u>T<sub>2</sub></u> |
|----------------------|----------------------|
| R(A)                 |                      |
| R(B)                 |                      |
|                      | R(A)                 |
|                      | R(B)                 |
|                      | w(B)                 |
| w(A)                 |                      |

To this example to convert this schedule into a serial schedule we must have to swap the R(A) operation of transaction T<sub>2</sub> with the w(A) operation of transaction T<sub>1</sub>. However we cannot swap these two operations because they are conflict operations, thus we can say that this given schedule is not conflict serializable.

② View Serializability:

view serializability is a process to find out the a given schedule is view serializable or not.

To check whether a given schedule is view serializable we need to check whether the given schedule

is view equivalent to its serial schedule. Let's take an example to understand what I mean by that.

Given schedule:

T<sub>1</sub>

R(X)

W(X)

T<sub>2</sub>

R(X)

W(X)

R(Y)

W(Y)

R(Y)

W(Y)

Serial Schedule of the above given schedule:

T<sub>1</sub>

R(X)

W(X)

R(Y)

W(Y)

T<sub>2</sub>

R(X)

W(X)

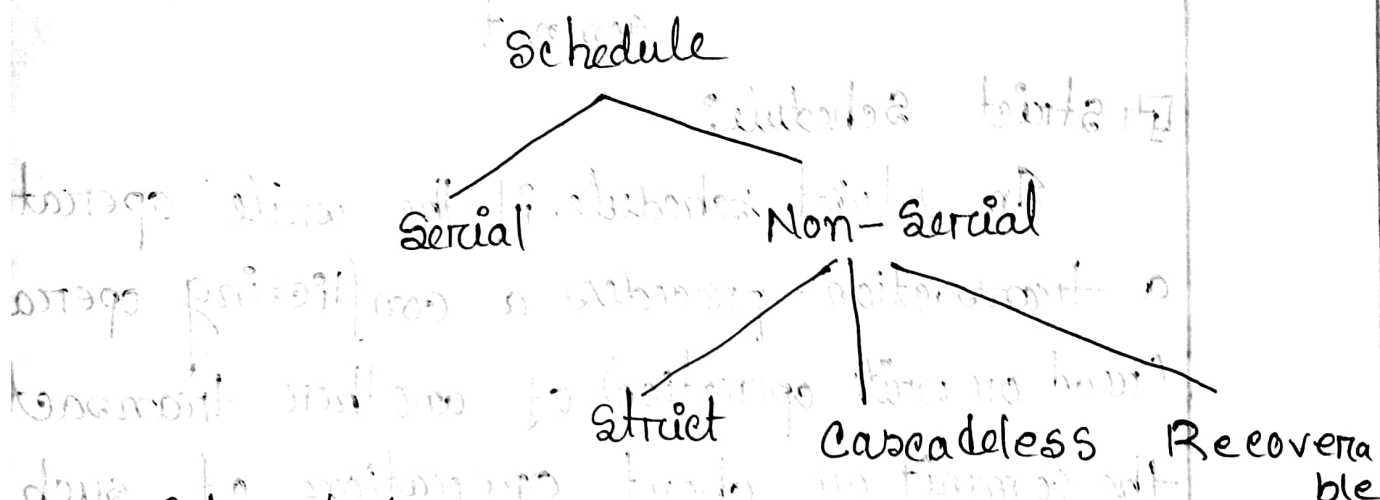
R(Y)

W(Y)

### ▣ Schedule:

We know that transaction are set of instructions and these instructions perform operation on database. When multiple transactions are running concurrently then there needs to be a sequence in which the operations are performed because at a time only one operation can be performed on the database. This sequence of operations is known as ~~se~~ schedule.

### ▣ Types of Schedules:



### ▣ Serial Schedule:

In serial schedule a transaction is executed completely before starting the execution of another transaction. This type of execution of transaction is also known as non-interleaved execution.

Ex: Here R refers to the read operation and W refers to the write operation. In this example

The transaction  $T_2$  does not start execution until the transaction  $T_1$  is finished.

|                         |                         |
|-------------------------|-------------------------|
| <u><math>T_1</math></u> | <u><math>T_2</math></u> |
| $R(A)$                  |                         |
| $R(B)$                  |                         |
| $w(A)$                  |                         |
| commit                  |                         |
|                         | $R(B)$                  |
|                         | $R(A)$                  |
|                         | $w(B)$                  |
|                         | commit                  |

### □ strict schedule:

In strict schedule if the write operation of a transaction precedes a conflicting operation (read or write operation) of another transaction then the commit or abort operation of such transaction should also precede the conflicting operation of other transaction.

Ex:  $T_a$        $T_b$

|        |               |
|--------|---------------|
| $R(X)$ | $R(X)$        |
| $w(X)$ |               |
| commit | $w(X)$        |
|        | $R(X)$ commit |

Here the write operation  $w(x)$  of  $T_a$  precedes the conflicting operation of  $T_b$  so the conflicting operation of  $T_b$  had to wait the commit operation of  $T_a$ .

☐ Cascadeless Schedule:

If a transaction is going to perform read operation on a value, it has to wait until the transaction who is performing write on that value commits.

Ex:

$T_a$

$R(x)$

$w(x)$

commit

$T_b$

$w(x)$

$R(x)$

$w(x)$

commit

☐ Recoverable Schedule:

If a transaction is reading a value which has been updated by some other transaction then this transaction can commit only after the commit of other transaction which is

updating value.

Ex:

$R(x)$

$W(x)$

$R(x)$

$W(x)$

$R(x)$

commit

commit

View Equivalent:

1. Initial Read: Initial read of each data item in transactions must match in both schedules.

2. Final Write: Final write operations on each data item must match in both the schedules.

3. Update Read: If in schedule  $S_1$ , the transaction  $T_1$  is reading a data item updated by  $T_2$  then in schedule  $S_2$ ,  $T_1$  should read the value after the write operation of  $T_2$  on same data item.

View Serializable:

If a schedule is view equivalent to its serial schedule then the given schedule is said to be view

## Serializable:

Ex:

Non - Serial

S1

T1

R(X)

W(X)

R(X)

W(X)

R(Y)

W(Y)

R(Y)

W(Y)

Serial

S2

T1

R(X)

W(X)

R(Y)

W(Y)

T2

R(X)

W(X)

R(Y)

W(Y)

Initial Read:

In schedule S1 transaction T1 first reads the data item X. In S2 also transaction T1 first reads the data item X.

Then check Y. In schedule S1 transaction T1 first reads the data item Y. In S2 also the first read operation on Y is performed by T1. We checked for both data items X & Y and the initial read condition is satisfied in S1 & S2.

Final Read:

In S1 the final W operation on X is done

by transaction T2. In S2 also transaction T2 performs the final write on x.

Let's check Y. In S1 the final write operation on Y is done by T2. In S2 final write on Y is done by T2.

We checked for both data items X & Y and the final write condition is satisfied in S1 & S2.

Update Read: In S1, T2 reads the value of X, written by T1. In S2 the same transaction T2 reads the X after it is written by T1.

In S1, transaction T2 reads the value of Y, written by T1. In S2 the same transaction T2 reads the value of Y after it is updated by T1.

The update read condition is also satisfied for both the schedules.

Results: Since all the three conditions that check whether the two schedules are view equivalent are satisfied in this example, which means S1 and S2 are view equivalent. Also as we know that the schedule of S1 is view serializable schedule.

Recoverability: It is a property of database system that ensures that, in the event of a failure or error, the system can recover the database to a consistent state.

There are several levels of recoverability that can be supported by a database system:

No-undo logging: This level of recoverability provides the ability to undo the effects of only guarantees that committed transaction are durable, but does not provide the ability to undo the effects of uncommitted transaction.

Undo logging: This level of recoverability provides the ability to undo the effects of uncommitted transaction but may result in the loss of updates made by committed transaction that occur after the failed transaction.

Redo-undo logging: This level of recoverability provides both undo and redo capabilities, ensuring that system can recover to a consistent state regardless of whether a transaction has been committed or not.

Redo logging: This level of recoverability provides the ability to redo the effects of committed transactions, ensuring that all committed updates are durable and can be recovered in the event of failure.

### Recoverable Schedules:

Schedules in which transactions commit only after all transactions whose changes they read commit are called recoverable schedules. In other words, if some transaction  $T_j$  is reading value update or written by some other transaction  $T_i$ , then the commit of  $T_j$  must occur after the commit of  $T_i$ .

Ex:

$S_1: R_1(x), w_1(x), R_2(x), R_1(y), R_2(y),$   
 $w_2(x), w_1(y), c_1, c_2;$

Given schedule follows order of  $T_i \rightarrow T_j \Rightarrow c_1 \rightarrow c_2$ . Transaction  $T_1$  is executed before  $T_2$  hence there is no chance of conflict occur.  $R_1(x)$  appears before  $w_1(x)$  and transaction  $T_1$  is committed before  $T_2$  i.e. completion of first transaction performed first update on data item  $x$ , hence given schedule is recoverable.

Irrecoverable schedule: The table below shows a schedule with two transactions, T1 reads and writes A and that value is read and written by T2. T2 commits. But later on, T1 fails. So, we have to rollback T1. Since T2 has read the value written by T1, it should also be rolled back. But we have already committed that. So this schedule is irrecoverable schedule.

| T <sub>1</sub> | T <sub>1</sub> 's buffer | T <sub>2</sub> | T <sub>2</sub> Buffer space | Database |
|----------------|--------------------------|----------------|-----------------------------|----------|
|                |                          |                |                             | A = 5000 |
| R(A)           | A = 5000                 |                |                             | A = 5000 |
| A = A - 100    | A = 4000                 |                |                             | A = 5000 |
| W(A)           | A = 4000                 |                |                             | A = 4000 |
|                |                          | R(A);          | A = 4000                    | A = 4000 |
|                |                          | A = A + 500    | A = 4500                    | A = 4000 |
|                |                          | W(A);          | A = 4500                    | A = 4000 |
|                |                          | Commit         |                             |          |
| Failure point  |                          |                |                             |          |
| Commit         |                          |                |                             |          |

☐ Recoverable with cascading Rollback: The table below shows a schedule with two transactions, T1 reads and writes A and that value is read and written by T2. But later on, T1 fails. So we have to rollback T1. Since T2 has read the value written

by  $T_1$ , it should also be rolled back. As it has not committed, we can roll back  $T_2$  as well. So it is recoverable with cascading rollback. Therefore, there is no  $T_1$  is reading value updated by  $T_1$  and commit of  $T_2$  is delayed till commit of  $T_1$  the schedule is called recoverable with cascading rollback.

| $T_1$         | $T_1$ 's Buffer Space | $T_2$         | $T_2$ 's Buffer Space | Data base  |
|---------------|-----------------------|---------------|-----------------------|------------|
|               |                       |               |                       | $A = 5000$ |
| $R(A)$        | $A = 5000$            |               |                       | $A = 5000$ |
| $A = A - 100$ | $A = 4000$            |               |                       | $A = 5000$ |
| $W(A)$        | $A = 4000$            |               |                       | $A = 4000$ |
|               |                       | $R(A)$        | $A = 4000$            | $A = 4000$ |
|               |                       | $A = A + 500$ | $A = 4500$            | $A = 4000$ |
|               |                       | $W(A)$        | $A = 4500$            | $A = 4500$ |
| Failure Point |                       |               |                       |            |
| Commit        |                       |               |                       |            |
|               |                       | Commit        |                       |            |

**Case 2: Recoverable Roll-back:** The table below shows a schedule with two transactions.  $T_1$  reads and writes  $A$  and commits and that value is read by  $T_2$ . But if  $T_1$  fails before commit, no other transaction has

read its value, so there is no need to rollback other transaction. So, this is a cascadeless recoverable schedule.

☐ The capabilities of recoverability in a DBMS encompass:

**Atomicity:** Transaction in a DBMS are designed to be atomic, which means that they either entire absolutely or are rolled back to their unique nation in case of a failure. This guarantess that the database is usually in a consistent nation.

**Durability:** Once a transaction is dedicated its changes are permanently stored to the database even in the occasion of a failure. This ensaures that the database may be restored to its closing consistent kingdom after a failure.

**Logging:** A DBMS keeps a log of all transaction to make sure recoverability. The log consists of data about all adjustments made to the database, as well as the transactions that made those changes. In the event of a failure, the log may be used to repair the database to a regular state.

**Checkpointing:** A checkpoint is a point in time in which the DBMS records the consistency of the database and logs it. This allows lessening the quantity of time required for recovery in case of a failure, as handiest the transactions since the last checkpoint need to be rolled back or replayed.

**Recovery manager:** A restoration supervisor is a part of a DBMS that is accountable for restoring the database to a constant state after a failure. The healing supervisor uses the log and check points to determine which transactions need to be rolled returned or replayed to repair the database.

**Media recuperation:** Media recovery refers back to the capability of a DBMS to recover from a failure that affects the garage media, such as a hard disk crash. This includes restoring the database from a backup and making use of the log to drive deliver it up to date.

## ☐ Schedulers:

☐ A transaction that successfully completes its execution will have a commit instructions as the last statement.

☐ A transaction that fails to successfully complete its execution will have an abort instruction as the last statement.

### Schedule 1

☐ Let  $T_1$  transfer \$50 from A to B and  $T_2$  transfer 10% of the balance from A to B.

☐ An example of a serial schedule in which  $T_1$  is followed by  $T_2$

| $T_1$         | $T_2$             |
|---------------|-------------------|
| read (A)      |                   |
| $A := A - 50$ |                   |
| write (A)     |                   |
| read (B)      |                   |
| $B := B + 50$ |                   |
| write (B)     |                   |
| commit        |                   |
|               | read (A)          |
|               | $temp := A * 0.1$ |
|               | $A := A - temp$   |
|               | write (A)         |
|               | read (B)          |
|               | $B := B + temp$   |
|               | write (B)         |
|               | commit            |

## Schedule 2

A serial schedule in which  $T_2$  is followed by  $T_1$ :

| $T_1$                                                                                                                               | $T_2$                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{read}(A)$<br>$A := A - 50$<br>$\text{write}(A)$<br>$\text{read}(B)$<br>$B := B + 50$<br>$\text{write}(B)$<br>$\text{Commit}$ | $\text{read}(A)$<br>$\text{temp} := A * 0.1$<br>$A := A - \text{temp}$<br>$\text{write}(A)$<br>$\text{read}(B)$<br>$B := B + \text{temp}$<br>$\text{write}(B)$<br>$\text{Commit}$ |

## Schedule 3

Let  $T_1$  and  $T_2$  be the transactions defined previously. The following schedule is not a serial schedule, but it is equivalent to schedule 1.

| $T_1$                                                                     | $T_2$                                                                                       |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| $\text{read}(A)$<br>$A := A - 50$<br>$\text{write}(A)$                    | $\text{read}(A)$<br>$\text{temp} := A * 0.1$<br>$A := A + \text{temp}$<br>$\text{write}(A)$ |
| $\text{read}(B)$<br>$B := B + 50$<br>$\text{write}(B)$<br>$\text{commit}$ | $\text{read}(B)$<br>$B := B + \text{temp}$<br>$\text{write}(B)$<br>$\text{commit}$          |

Note - In schedules 1, 2 and 3 the sum " $A+B$ " is preserved.

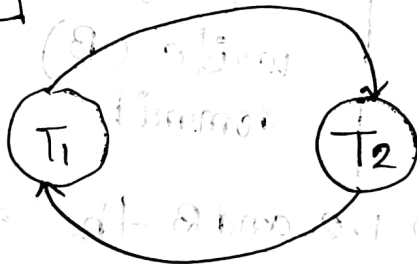
### □ Simplified view of transactions:

- We ignore operation other than read and write instructions.
- We assume that transactions may perform arbitrary computation on data in local buffers in between reads and writes.
- Our simplified schedules consists of only read and write instructions.

## ☐ Precedence Graphs:

- Consider some schedule of a set of transactions  $T_1, T_2, \dots, T_n$
- Precedence graph - a directed graph where the vertices are the transactions (names)
- We draw an arc from  $T_i$  to  $T_j$  if the two transaction conflict, and  $T_i$  accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.

Ex:

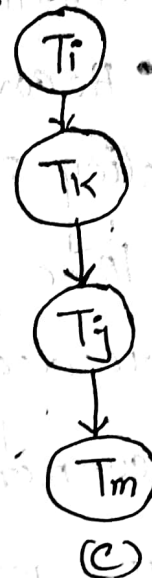
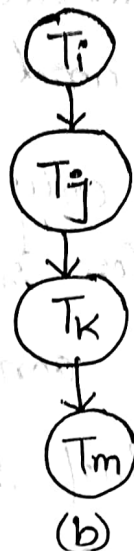
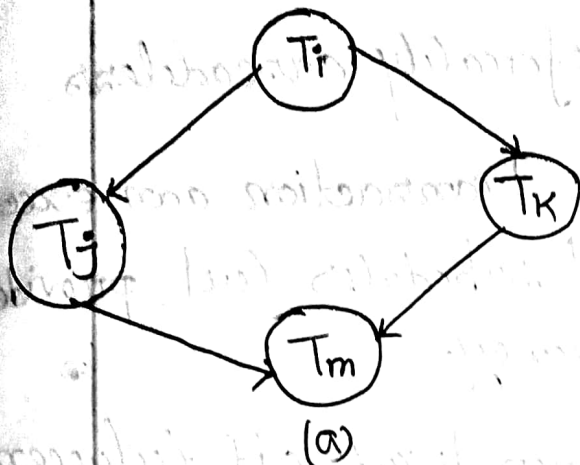


## ☐ Testing for Conflict Serializability:

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithm exist which take order  $n^2$  time, where  $n$  is the number of vertices in the graph.
- If precedence graph is acyclic, the serializability order can be obtained by a topological sorting of the graph.

- That is a linear order, consistent with the partial order of the graph.

- For example, a serialization order for the schedule (a) would be one of either (b) or (c)



□ **Cascadeless Schedules:** for each pair of transaction  $T_i$  and  $T_j$  such that  $T_j$  reads a data item previously written by  $T_i$  - the appears before the read operation of  $T_j$ .

- Every cascadeless schedule is also recoverable.
- It is desirable to restrict the schedule to those that are cascadeless.
- Example of a schedule that is NOT cascadeless

| $T_{i0}$                       | $T_{i1}$            | $T_{i2}$ |
|--------------------------------|---------------------|----------|
| read(A)<br>read(B)<br>write(A) | read(A)<br>write(A) | read(A)  |
| abort                          |                     |          |

## Concurrency Control:

- A database must provide a mechanism that will ensure that all possible schedules are both:

- Conflict serializable

- Recoverable and preferably cascadeless

- A policy in which only one transaction can execute at a time generates serial schedules but provides a poor degree of concurrency.

- Concurrency control schemes trade off between the amount of concurrency they allow and the amount of overhead that they allow and the amount of overhead that they incur.

- Testing a schedule for serializability after it has executed is a little too late!

- Goal - to develop concurrency control protocols that will assure serializability.

## Weak levels of consistency:

- Some applications are willing to live with weak levels of consistency allowing schedulers that are not serializable.
- Trade off accuracy for performance.

## Levels of consistency in SQL-92:

- Serializable - default.
- Repeatable read - only committed records to be read, repeated reads of same record must return same value. However, a transaction may not be serializable - it may find some records inserted by a transaction but not find others.
- Read committed: only committed records can be read but successive reads of record may return different values.
- Read uncommitted: even uncommitted records may be read.

Lower degree of consistency useful for gathering approximate information about the database.

Warning: some database systems do not ensure

serializable, schedules by default.

Transaction Definition in SQL:

■ Data manipulation language must include a construct for specifying the set of actions that comprise a transaction.

■ In SQL, a transaction begins implicitly.

■ A transaction in SQL ends by:

- Commit work: commits current transaction and begins a new one.

- Rollback work: causes current transaction and to abort.

■ In almost all database systems, by default every SQL statement also commits implicitly if it executes successfully.